



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 4
по курсу «Компьютерные сети»
«HTTP прокси сервер»

Студент группы ИУ9-32Б Ивенкова О. В.

Преподаватель Посевин Д. П.

Москва 2024

1 Задание

Реализовать HTTP-прокси-сервер на языке Go, который позволит пользователю вводить URL-адрес в веб-интерфейсе, после чего сервер запрашивает содержимое по этому URL-адресу, обрабатывает его и возвращает пользователю.

2 Результаты

Исходный код программы представлен в листинге 1.

Листинг 1: Реализация файла peer1.go

```
1 package main
2
3 import (
4     "fmt"
5     "golang.org/x/net/html"
6     "io"
7     "io/ioutil"
8     "log"
9     "net/http"
10    "os"
11    "path/filepath"
12    "strings"
13 )
14
15 var (
16     LOCAL_HREF = "http://185.102.139.169:5420/?url="
17     LOCAL_SRC  = "src=\"http://185.102.139.169:5420/static/"
18 )
19
20 func Handler(w http.ResponseWriter, r *http.Request) {
21     url := r.FormValue("url")
22     if url == "" {
23         http.ServeFile(w, r, "index.html")
24     } else {
25
26         client := &http.Client{}
27         req, err := http.NewRequest("GET", url, nil)
28         if err != nil {
29             log.Fatal(err)
30         }
31     }
```

```

32     resp, err := client.Do(req)
33     if err != nil {
34         log.Fatal(err)
35     }
36     defer resp.Body.Close()
37
38     if resp.StatusCode != http.StatusOK {
39         http.Error(w, "Failed to load the page", http.StatusBadGateway)
40         return
41     }
42
43     body, err := ioutil.ReadAll(resp.Body)
44     if err != nil {
45         log.Fatal(err)
46     }
47
48     contentType := resp.Header.Get("Content-Type")
49     if strings.HasPrefix(contentType, "text/html") {
50         // Parsing HTML and downloading resources
51         modifiedHTML := ParseAndDownloadResources(url, string(body))
52         w.Header().Set("Content-Type", contentType)
53         fmt.Fprintf(w, modifiedHTML) // Return modified HTML
54     } else {
55         // Simply pass non-HTML content
56         w.Header().Set("Content-Type", contentType)
57         w.Write(body)
58     }
59 }
60 }
61
62 // Function to parse HTML and download resources
63 func ParseAndDownloadResources(baseUrl, body string) string {
64     tokenizer := html.NewTokenizer(strings.NewReader(body))
65     var modifiedHTML strings.Builder
66
67     for {
68         tt := tokenizer.Next()
69         switch tt {
70             case html.ErrorToken:
71                 if tokenizer.Err() == io.EOF {
72                     return modifiedHTML.String() // Return assembled HTML
73                 }
74                 log.Fatal(tokenizer.Err())
75             case html.StartTagToken, html.SelfClosingTagToken:
76                 t := tokenizer.Token()
77                 modifiedHTML.WriteString("<" + t.Data) // Start of the tag

```

```

78
79     for _, attr := range t.Attr {
80         attrStr := fmt.Sprintf(' %s="%s"', attr.Key, attr.Val)
81         if (t.Data == "img" && attr.Key == "src") ||
82             (t.Data == "link" && attr.Key == "href" && strings.HasSuffix(
attr.Val, ".css")) ||
83             (t.Data == "script" && attr.Key == "src") {
84
85             // Process src and href for images, styles, and scripts
86             resourceURL := ResolveURL(baseURL, attr.Val)
87             filePath := DownloadAndCacheResource(resourceURL)
88
89             if filePath != "" {
90                 attrStr = fmt.Sprintf(' %s="/static/%s"', attr.Key, filePath
) // Update link to local
91             }
92         } else if t.Data == "a" && attr.Key == "href" { // Process <a>
links
93             // Redirect all links through the local server
94             newHref := ResolveURL(baseURL, attr.Val)
95             attrStr = fmt.Sprintf(' %s="%s%s"', attr.Key, LOCAL_HREF,
newHref)
96         }
97         modifiedHTML.WriteString(attrStr) // Add tag attributes
98     }
99
100     if t.Data[len(t.Data)-1] == '/' {
101         modifiedHTML.WriteString("/>") // Close the tag
102     } else {
103         modifiedHTML.WriteString(">") // Open the tag
104     }
105     case html.TextToken:
106         modifiedHTML.Write(tokenizer.Text()) // Add text inside tags
107     case html.EndTagToken:
108         t := tokenizer.Token()
109         modifiedHTML.WriteString("</" + t.Data + ">") // Close the tag
110     }
111 }
112 return modifiedHTML.String() // Return modified HTML
113 }
114
115 // Function to download a resource and save it locally
116 func DownloadAndCacheResource(url string) string {
117     fileName := filepath.Base(url)
118     filePath := filepath.Join("static", fileName)
119

```

```

120 // Check if the file exists
121 if _, err := os.Stat(filePath); os.IsNotExist(err) {
122     // If the file doesn't exist, download it
123     resp, err := http.Get(url)
124     if err != nil {
125         log.Println("Failed to download resource:", err)
126         return ""
127     }
128     defer resp.Body.Close()
129
130     out, err := os.Create(filePath)
131     if err != nil {
132         log.Println("Failed to save resource:", err)
133         return ""
134     }
135     defer out.Close()
136
137     _, err = io.Copy(out, resp.Body)
138     if err != nil {
139         log.Println("Failed to copy resource to file:", err)
140         return ""
141     }
142 }
143 return fileName
144 }
145
146 // Function to correctly create a full URL
147 func ResolveURL(baseURL, relativeURL string) string {
148     if strings.HasPrefix(relativeURL, "http") {
149         return relativeURL
150     }
151     if strings.HasPrefix(relativeURL, "/") {
152         // If the link is relative and starts with "/", add protocol and
153         // domain
154         base := baseURL[:strings.Index(baseURL, "//")+2] + strings.Split(
155             baseURL, "/")[2]
156         return base + relativeURL
157     }
158     return baseURL + relativeURL
159 }
160
161 func main() {
162     // Create a folder for storing downloaded resources if it doesn't
163     exist
164     os.Mkdir("static", 0755)

```

```

163 http.HandleFunc("/", Handler)
164 http.Handle("/static/", http.StripPrefix("/static/", http.FileServer(
    http.Dir("static"))))
165 log.Fatal(http.ListenAndServe(":5420", nil))
166 }

```

Результат запуска представлен на рисунке 1.



Рис. 1 — Результат